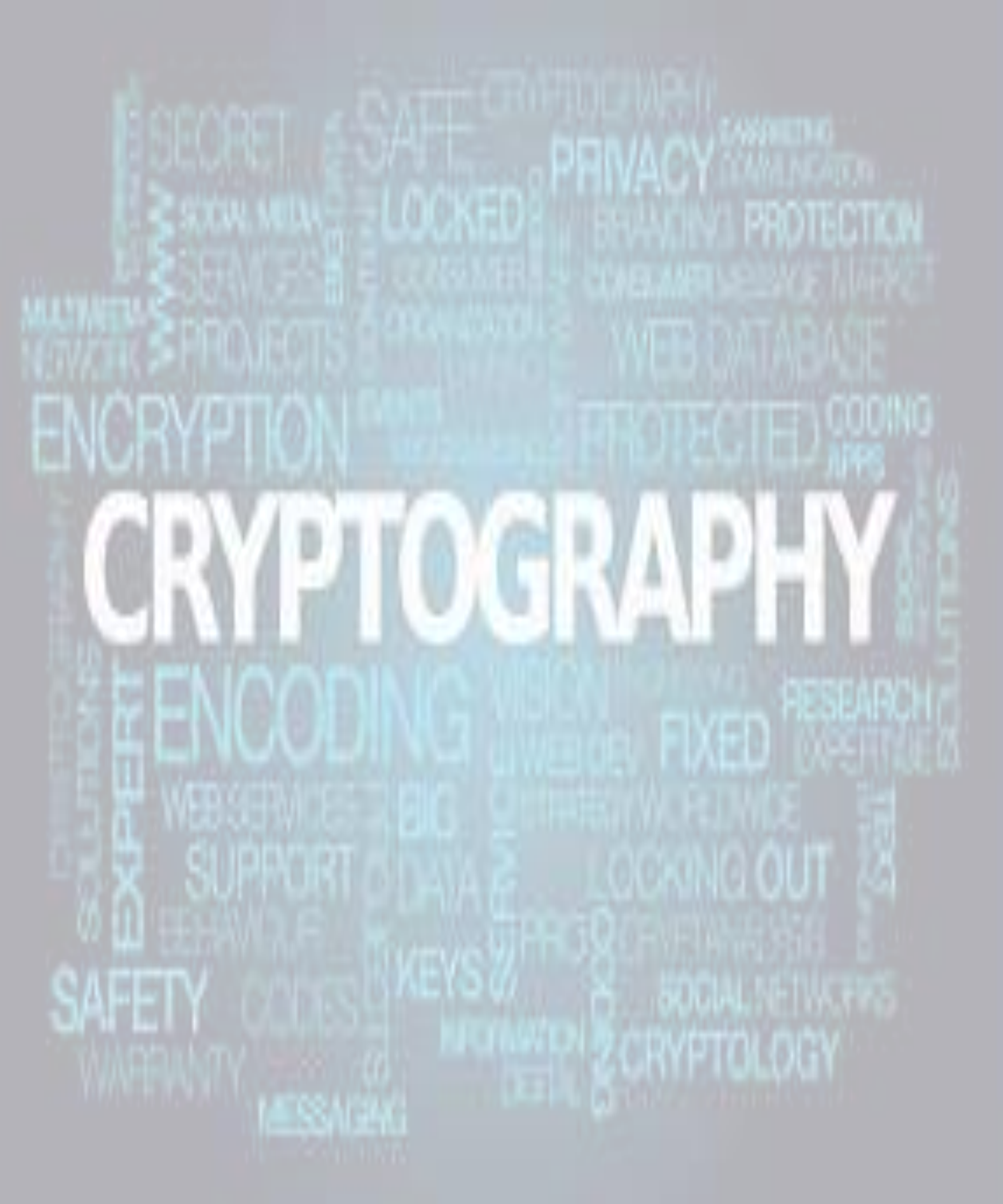

CRYPTOGRAPHY
SECRET
SAFE
PRIVACY
LOCKED
PROTECTION
ENCRYPTION
ENCODING
FIXED
SUPPORT
LOCKING OUT
SAFETY
CRYPTOLOGY

01 Hashing

02 Keys / Secret Keys

03 Symmetric Cryptography

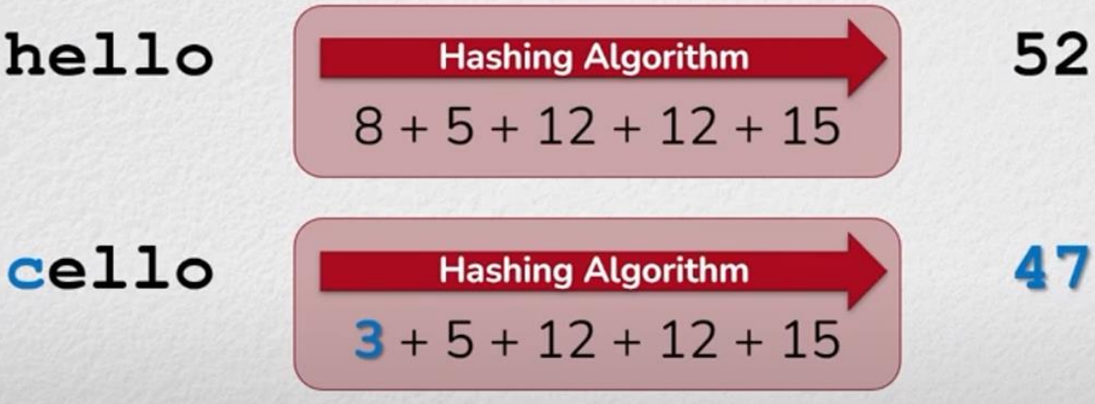
- 04 Encryption
- 05 MAC / HMAC
- 06 Pseudo Random Function

07 Asymmetric Cryptography

- 08 Asymmetric Encryption (RSA)
- 09 Signatures
  - 10 RSA Signatures
  - 11 DSA Signatures
- 12 Key Exchanges
  - 13 RSA Key Exchanges
  - 14 DH Key Exchanges
- 15 Elliptic Curve Cryptography

Hashing

- Mathematical formulas that transform messages into **deterministic, fixed-length** representational strings
- **Purpose**: Has original message changed since it was hashed?



- Impossible to reverse – *but can be brute forced*
- Very efficient, suitable for bulk data hashing

To Hash:	
Input:	Message / Data / Packet
Output:	Digest

Legacy: **md5** **sha1**
Modern: SHA224 **SHA256** SHA384 SHA512
Future: SHA3-224 SHA3-256 SHA3-384 SHA3-512

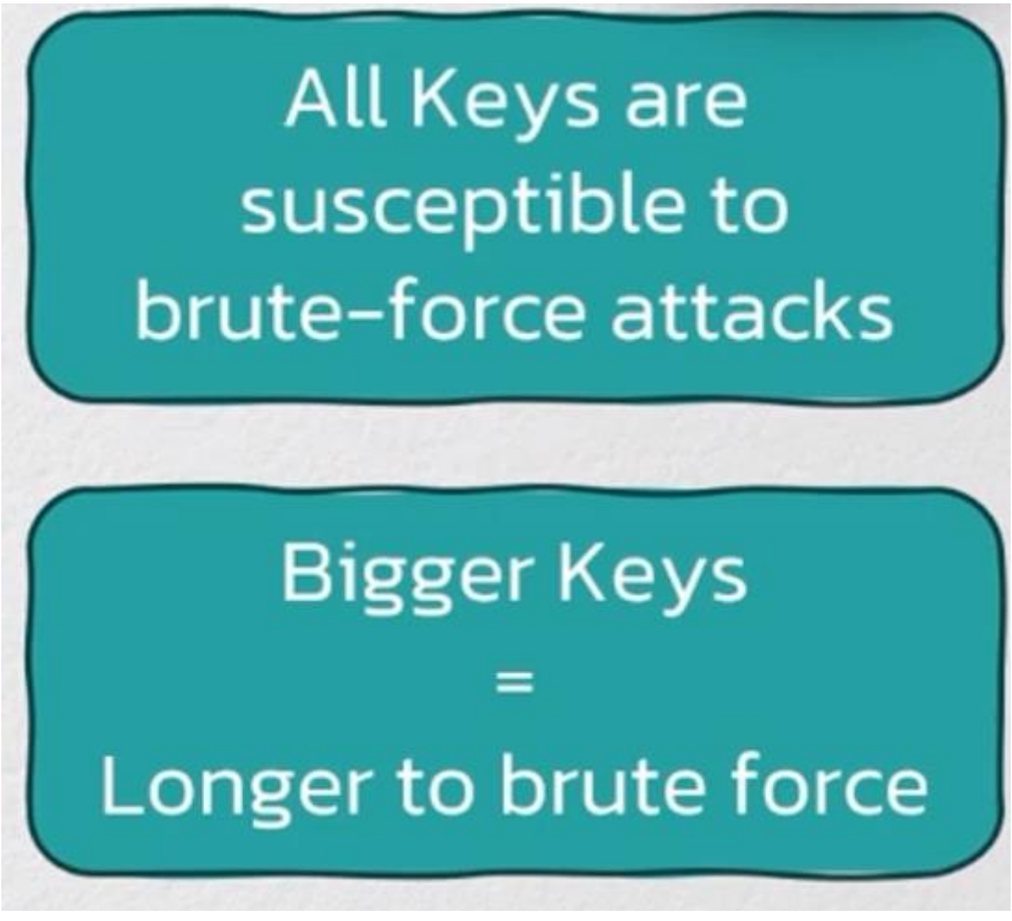
```
$
$ sha1sum file*
5ea90a12fed5c6849bc16464fd221a39e56927f9  file1.txt
f0f72912a9b0cc04a452d16022a81e85c52b1253  file2.txt
e67cc73db7e1762ef2168fddfeebda543d09fdb7  file3.txt
$
```

Keys / Secret Keys

- Sequence of 1's and 0's used in Cryptography

```
1101001001101111100100010010101010000001000000101111100000100001
```

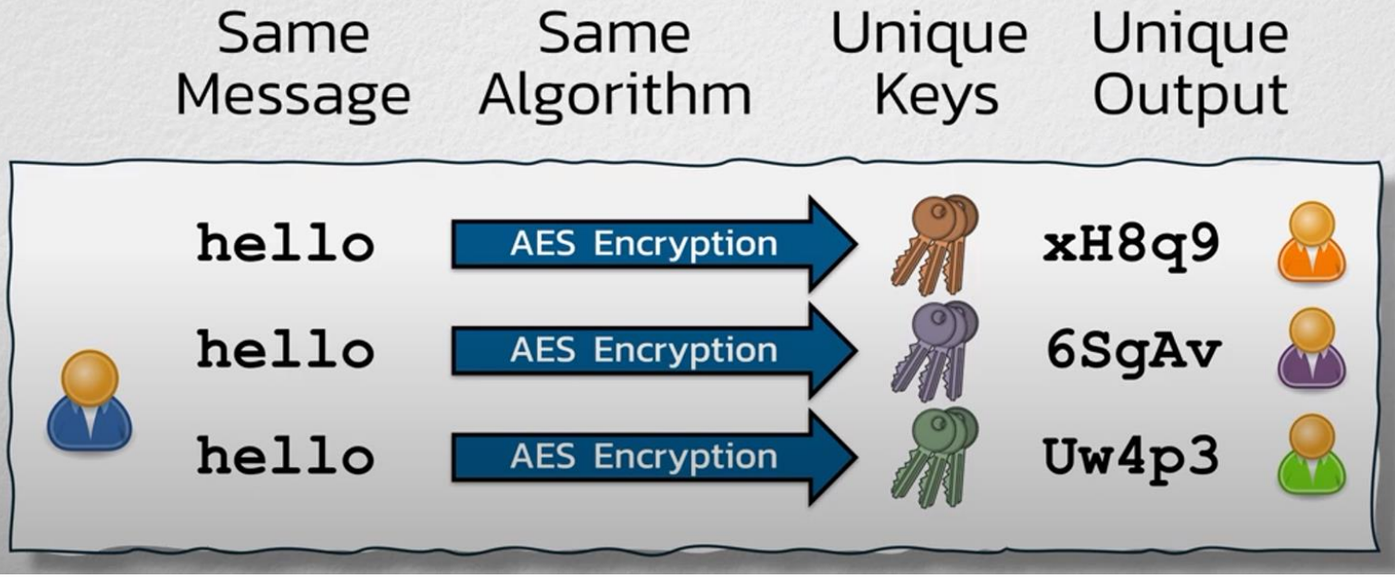
  - Sized in numbers of "bits"
    - 1 bit _ 1 or 0
    - 2 bits __ 00 01 10 11
    - 3 bits ___ 000 001 010 011 100 101 110 111
    - 64 bits 64 digit string of 1's and 0's
      - 2^64 = 18,014,398,509,481,984 combinations



Why longer key hash takes longer to decrypt?

Keys / Secret Keys

- *Purpose*:
  Combine **industry-established algorithms** with **uniquely-generated keys**



- *Purpose*:
  Combine **industry-established algorithms** with **uniquely-generated keys**
  - Keys used by these algorithms can be Symmetric or Asymmetric

Symmetric Crypto – Encryption, MAC / HMAC, PRF

- **Same** keys to **perform** and **reverse or verify** an operation

Asymmetric Crypto – Encryption, Signatures, Key Exchanges

- **Different** keys to **perform** and **reverse/verify/complete** an operation
  - Keys are *different*... but *related* – generated as a "Key Pair"

Symmetric Key Cryptography

- Mathematical operations that are:
  - Performed with a secret key
  - Verified or Undone with the *identical* secret key

- Symmetric Cryptography includes *three* operations:
  - Symmetric Encryption
  - Message Authentication Codes (MACs)
  - Pseudo Random Functions (PRFs)

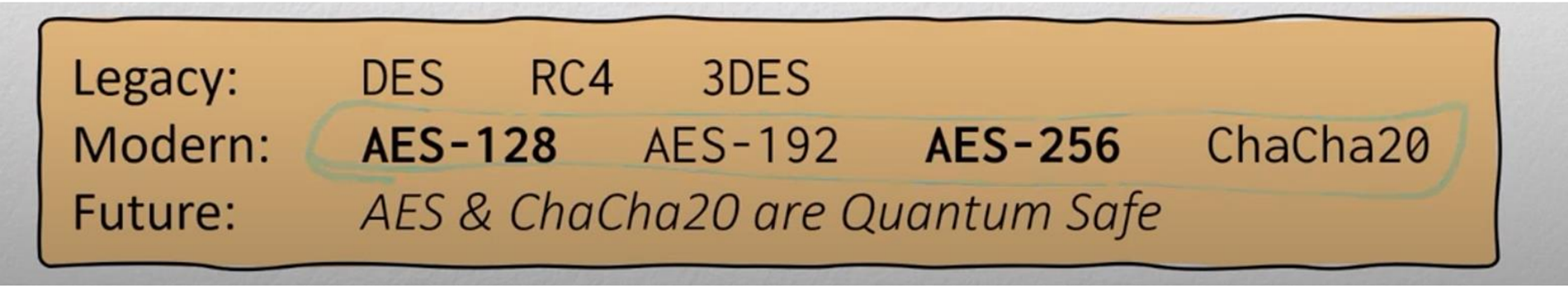
- Transforms plain readable text into something unintelligible
- Subsequently reverts it back to its original form

- Purpose: Only whoever has Secret Key can read original message
  - Confidentiality

$6 \times 7 = 42 \div 7 = 6$

- High efficiency, Suitable for Bulk Data

Symmetric Key Cryptography

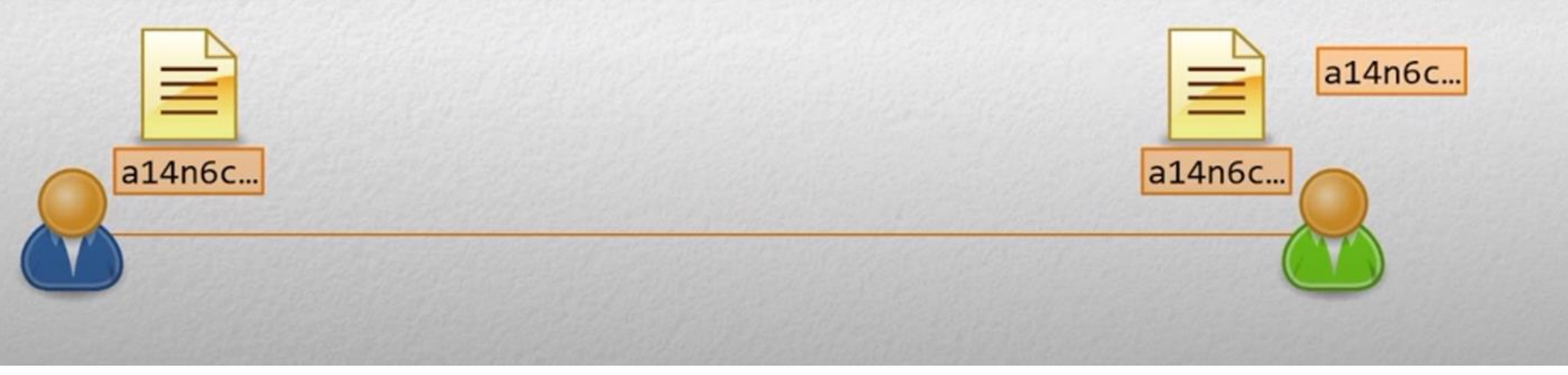


Note: Quantum computers are invented which can decrypt many types of traffic but not all.

HASHING

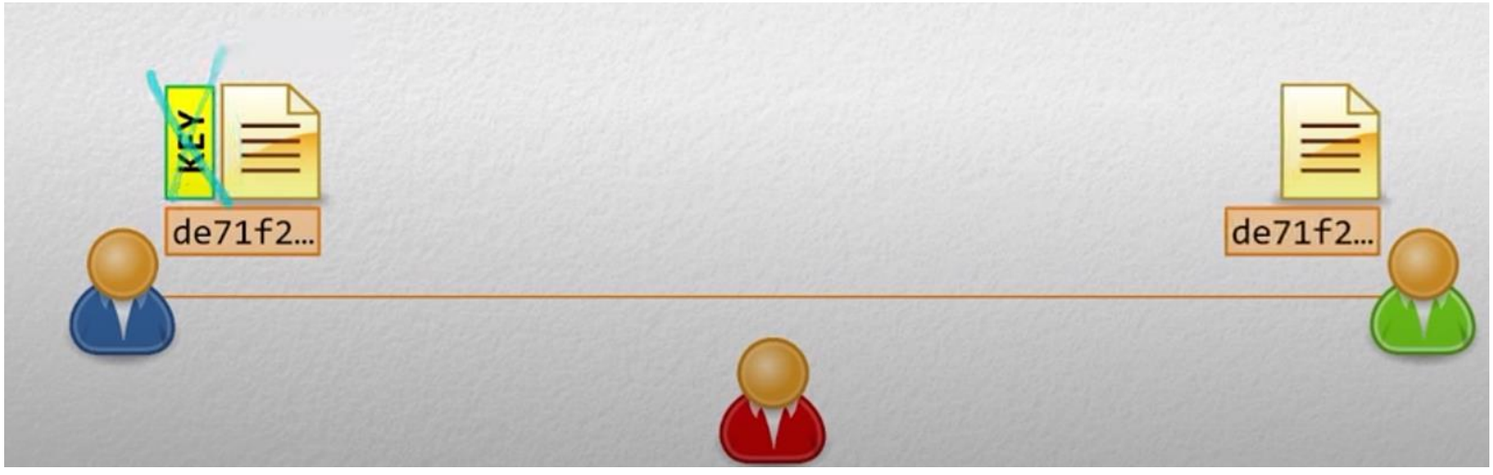
- Message Authentication Code
  - Concept of combining a **message** with a **secret key** before hashing
- **Purpose**: Detect unauthorized alteration of message & digest
  - Hashing alone is not enough

Always not true, why?



Here the hash value generated is changed in the middle by an actor while transition of data.

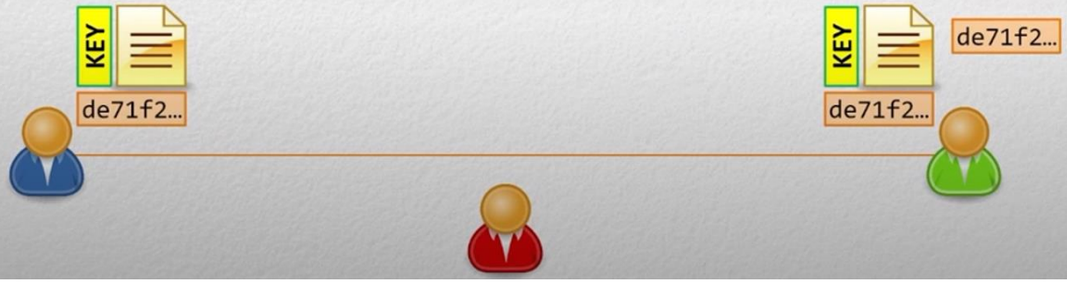
How does MAC Solved the Problem



Key won't be shared with the file. The user also has the copy of the key, with help of the key, the user can calculate the hash value by combining with message.

If the hash generated is similar to present in the message-digest then the message is not altered otherwise it is considered as an altered message.

- Message Authentication Code
  - Concept of combining a **message** with a **secret key** before hashing
- **Purpose**: Detect unauthorized alteration of message & digest
  - Only whoever has Secret Key can create an acceptable digest
  - **Integrity** & **Authentication** of bulk data

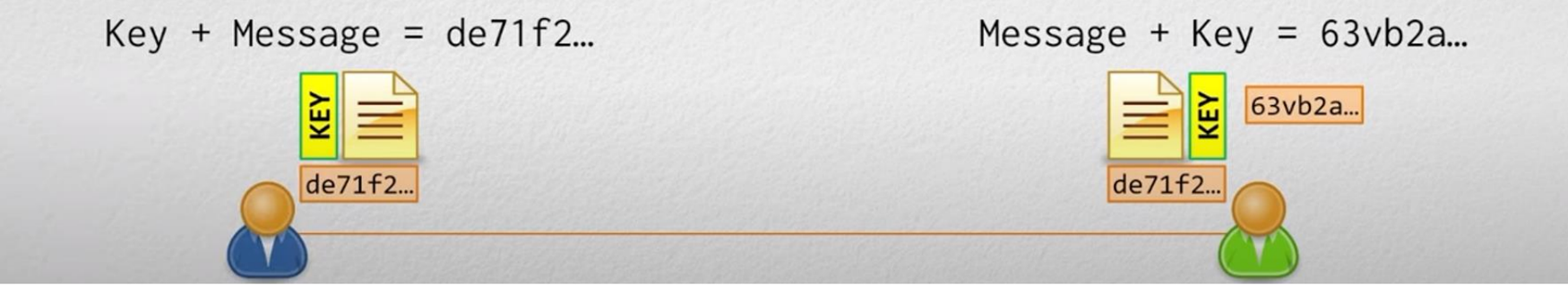


How does HMAC Solved the Problem

HAMC provides not only Mac specification but also some order to digest or hash the message.
Hash(A+ mess(B)) = C != hash(Mess(B)+A) != C
(hash(A+B)=hash(A)+hash(B))

- Message Authentication Code
  - Concept of combining a **message** with a **secret key** before hashing
- Hash-based Message Authentication Code (HMAC)
  - Industry standard way of combining message + secret key

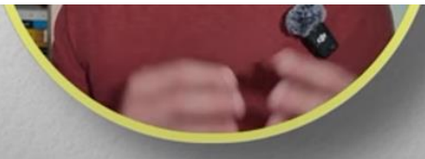
RFC 2104



To create a MAC:

Input: Message, Secret Key

Output: Message Authentication Code



(sometimes still called Digest)

Legacy:	*(none in significant use)*		
Modern:	**HMAC**	Poly1305	
Future:	GCM	CCM	*(AEAD Ciphers)*

Pseudo Random Function

- **Pseudo Random Function (PRF):**
  - Mathematical formula that transform "messages" into **deterministic**, **arbitrary-length** "representational strings"
- **Purpose:** Generate unlimited keys from a single secret key
  - Key Exchanges are expensive…
  - Do it once, then use PRF to generate any number of secret keys

Pseudo Random Function (PRF)	
Input:	**Secret, Desired # of Bits, Seed Data**
Output:	**Pseudo Random string of 1s and 0s**

- Secret – something known only to intended parties
- # of Bits – PRFs are run until this many bits output is attained

- **Key Derivation Function (KDF)**
  - Similar to PRF: specific input data = consistent output
  - Requires **Salt**, and include a "**slow down**" mechanism

- Additional random data "mixed in"
- Guarantees entropy

- # of iterations
- Intentionally memory intensive
- Intentionally prevent parallelization

Pseudo Random Function

- Purpose of a KDF: Makes brute force infeasible
  - Example: **Password Storage**
    - Salt guarantees minimum password length & complexity
    - Hashing + PRFs can be performed *billions / trillions* of times per second
  - KDF that slows to 0.1 seconds = 3000+ years for 1 trillion guesses

Finally:

- Hashing Algorithm:
  - Formula that transform input into **deterministic**, fixed-length representational strings
- Pseudo Random Function (PRF):
  - Transforms a **Secret** & **Label** into **deterministic**, arbitrary-length value indistinguishable from random data
  - Key Derivation Function (KDF)
    - PRF but more secure & more computationally expensive

Asymmetric Cryptography

- Mathematical operations that can be performed with *one* key, and verified or undone with *another* key
  - One key is kept secret
    "the private key"
  - Other key is shared freely
    "the public key"

A "key" does not always mean a *single* value

Public Key

Private Key

is whatever values are needed to

perform

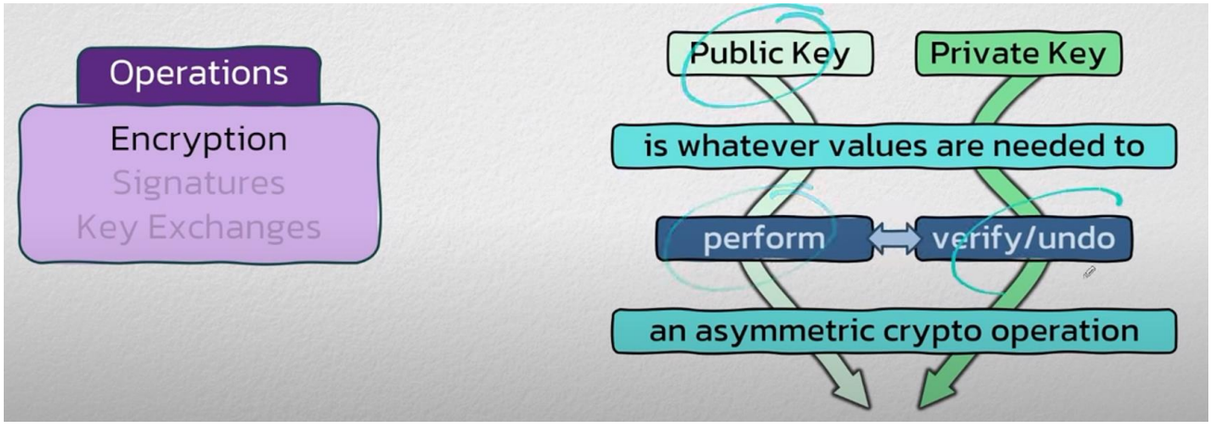
verify/undo

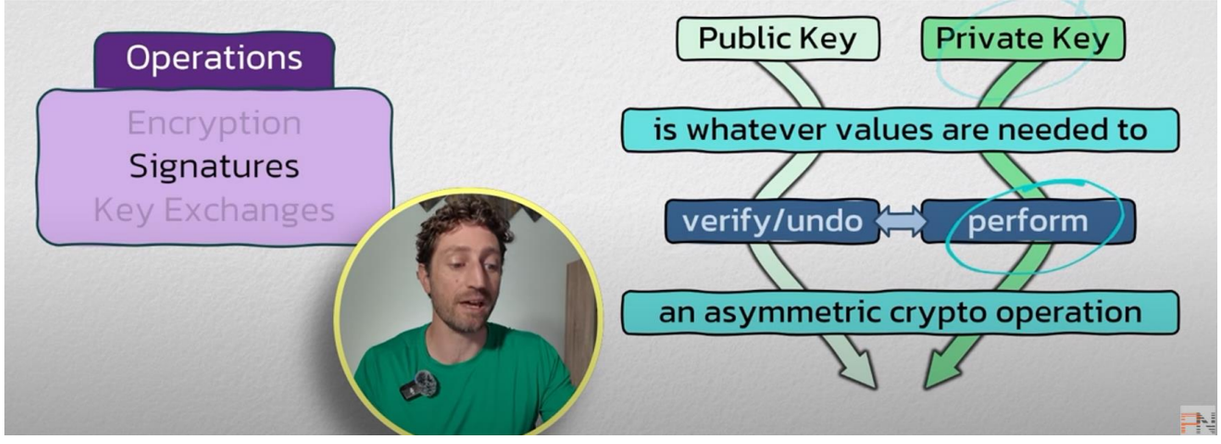
an asymmetric crypto operation

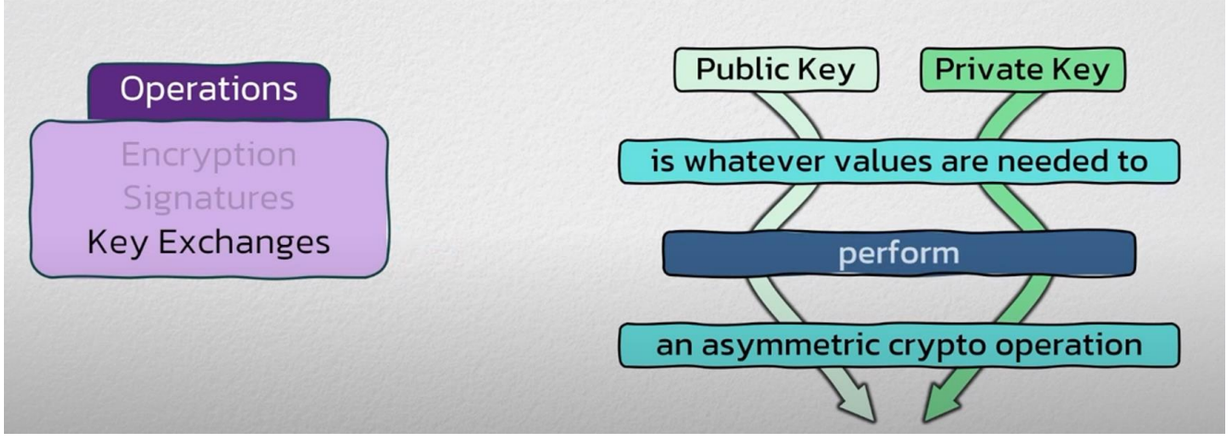
Client-Server architecture

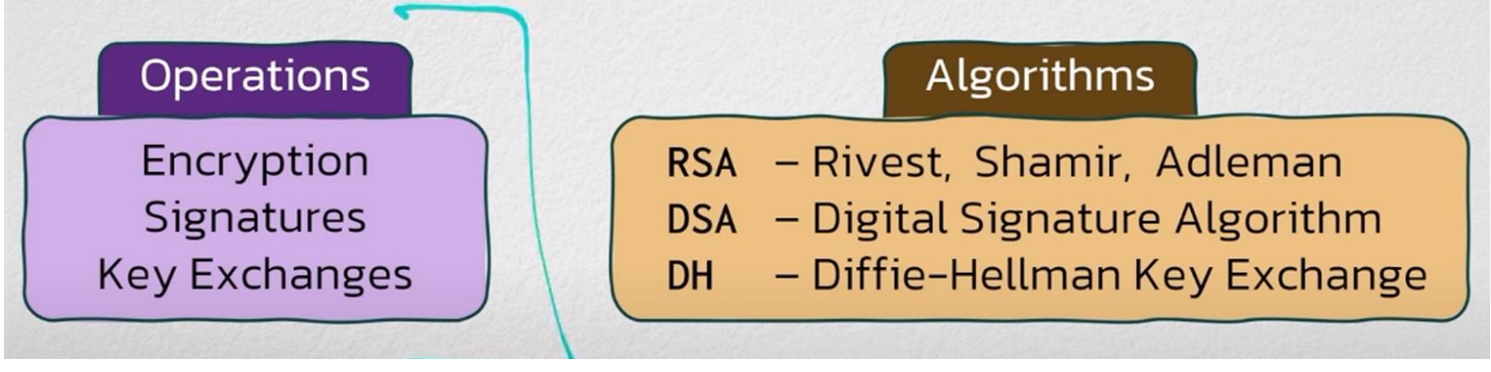
Vice-Versa

Asymmetric Cryptography









Asymmetric Cryptography

- Definition of **Encryption**:
  - Transforms plain readable data into something unintelligible
  - Subsequently reverts it back to its original form

- Asymmetric encryption uses *two* different keys
  - ***Public Key*** **for encryption**, ***Private Key*** **for decryption**
- Purpose: Only whoever has Private Key can read original message
  - **Confidentiality**

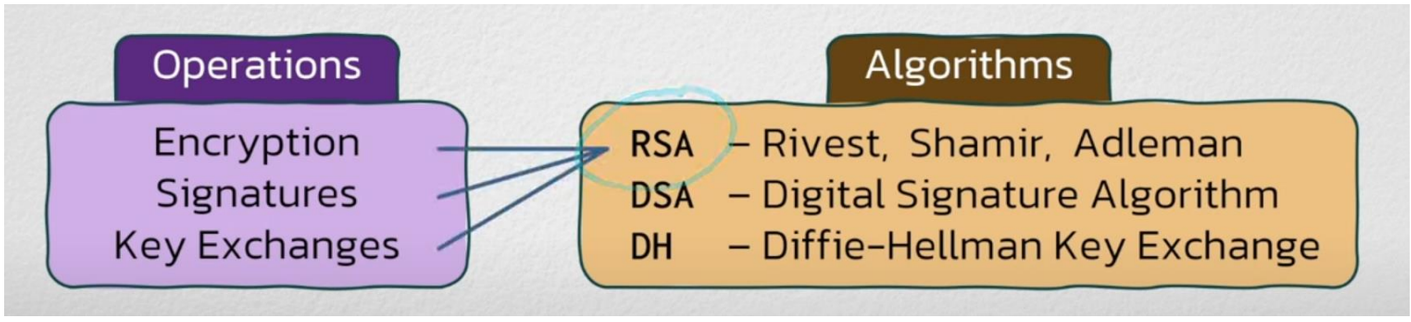
- Only **RSA** algorithm can be used for Asymmetric Encryption
  - Low efficiency – only suitable for limited, smaller data sets

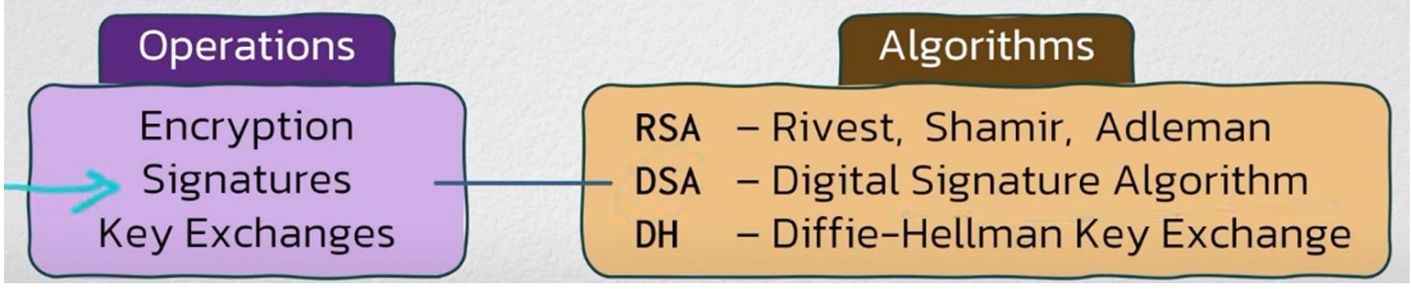
- RSA is "commutative" – math works in both directions
  - No Confidentiality – anyone can decrypt with Public Key
  - This property can be used for another operation …

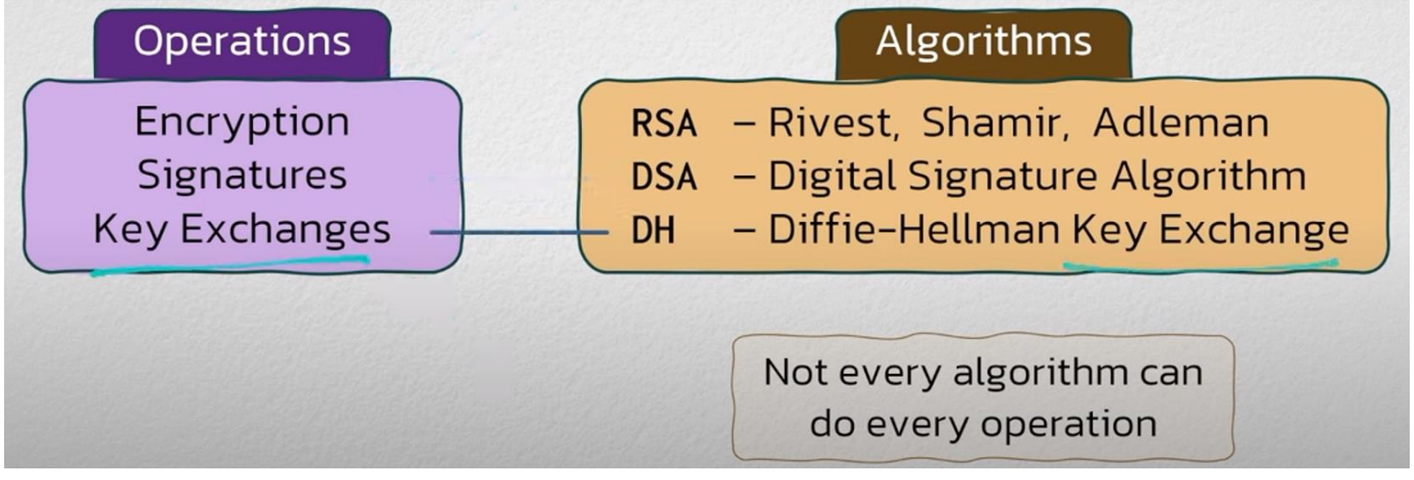
RSA Encryption	
Input:	**Plaintext**, **Public Key**
Output:	**Cipher Text**

RSA Decryption	
Input:	**Cipher Text**, **Private Key**
Output:	**Plaintext**

Asymmetric Cryptography

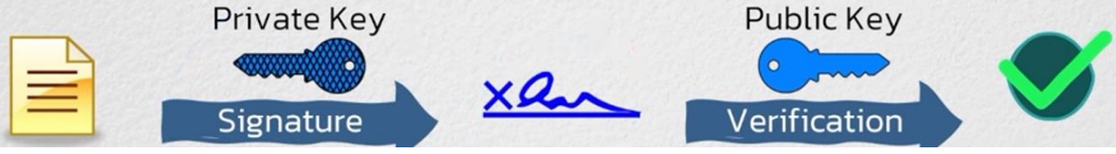






Signatures

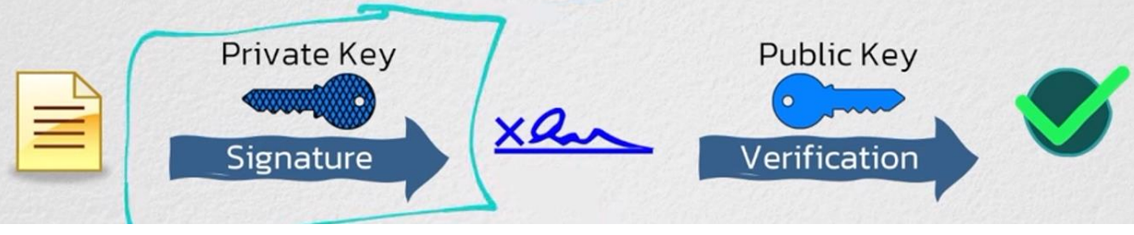
- Operation that guarantees data has not changed since it was signed
  - Key used for signing must only be known by the signer – ***Private Key***
  - Verification key must be accessible by everyone – ***Public Key***



- Nearly anything can be signed:
  - Documents / Emails / Files
  - Software
  - Certificates

Signatures are computationally expensive

- Operation that guarantees data has not changed since it was signed
  - Provides **Integrity** & **Authentication** of what is Signed



Integrity: If anyone changes the data then the signature can't be verified with the corresponding public key.

Authentication: The only person who can sign the file who has the private key for that particular public key.

- Differences between Signatures and MAC's:

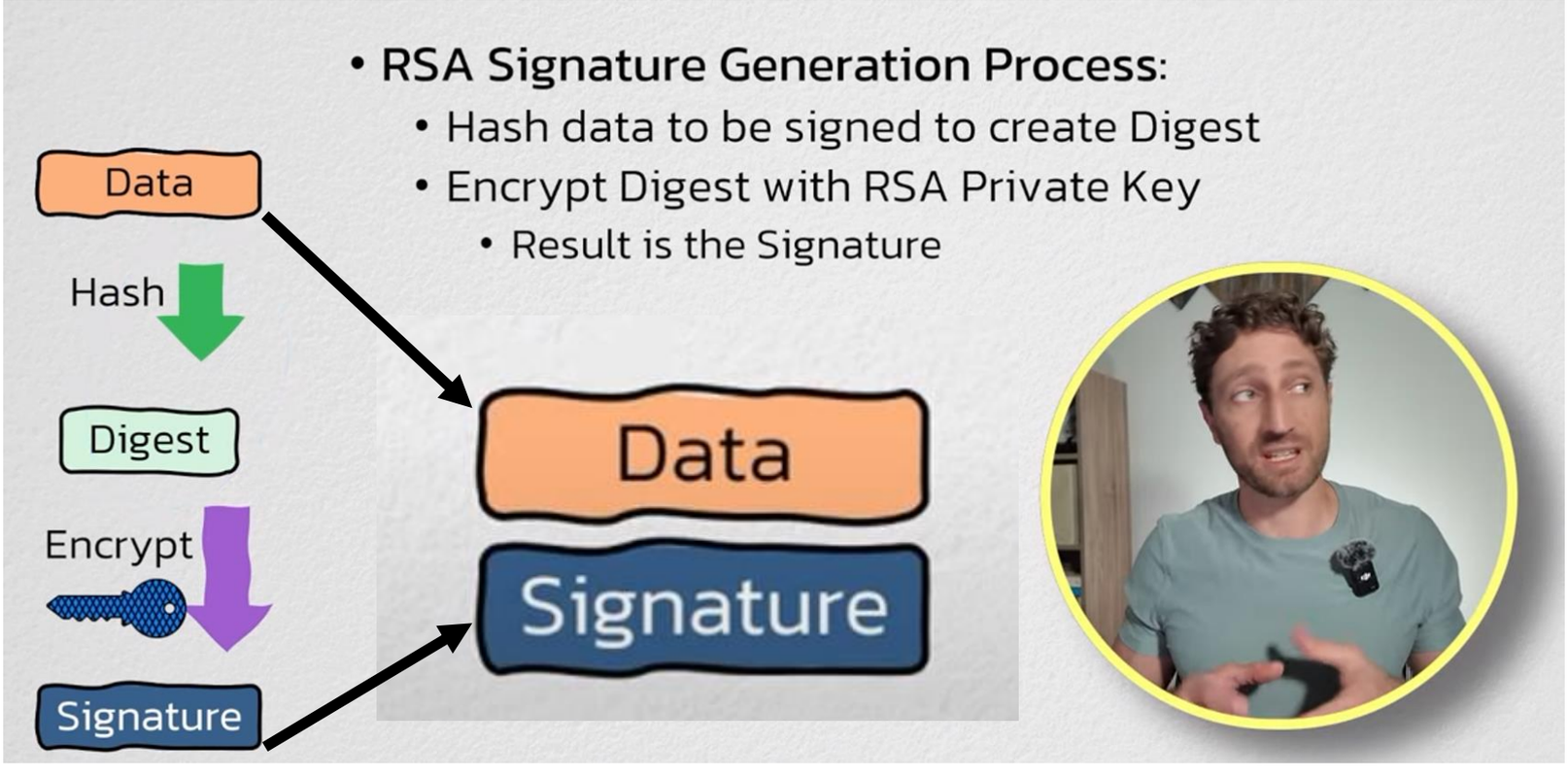
Signatures
- Asymmetric Operation – limited to smaller data
- Anyone can verify signature

MAC's
- Symmetrical Operation – suitable for bulk data
- Only peer can verify MAC

RSA – Rivest, Shamir, Adleman
DSA – Digital Signature Algorithm

RSA

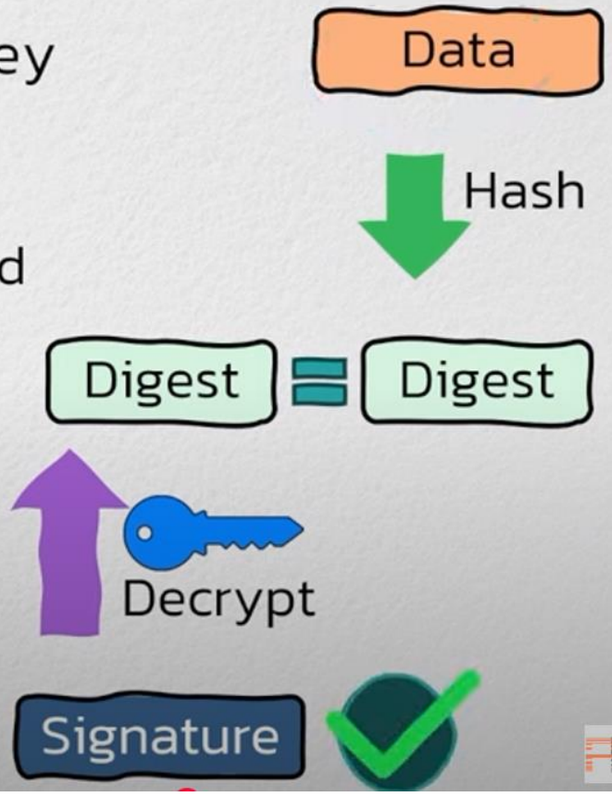
DEFI: Process to create and verify signatures takes advantage of RSA's ability to encrypt and decrypt in either direction.



RSA Verification

Step1: Separate Data and Signature

- **RSA Signature Verification Process:**
  - Recalculate the Hash on Data
  - Decrypt Signature with RSA Public Key
  - Compare results
    - If Digests match:
      Data not modified since it was signed

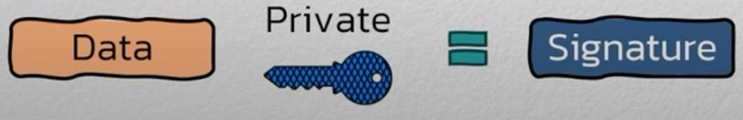


DSA (Digital Signature Algorithm)

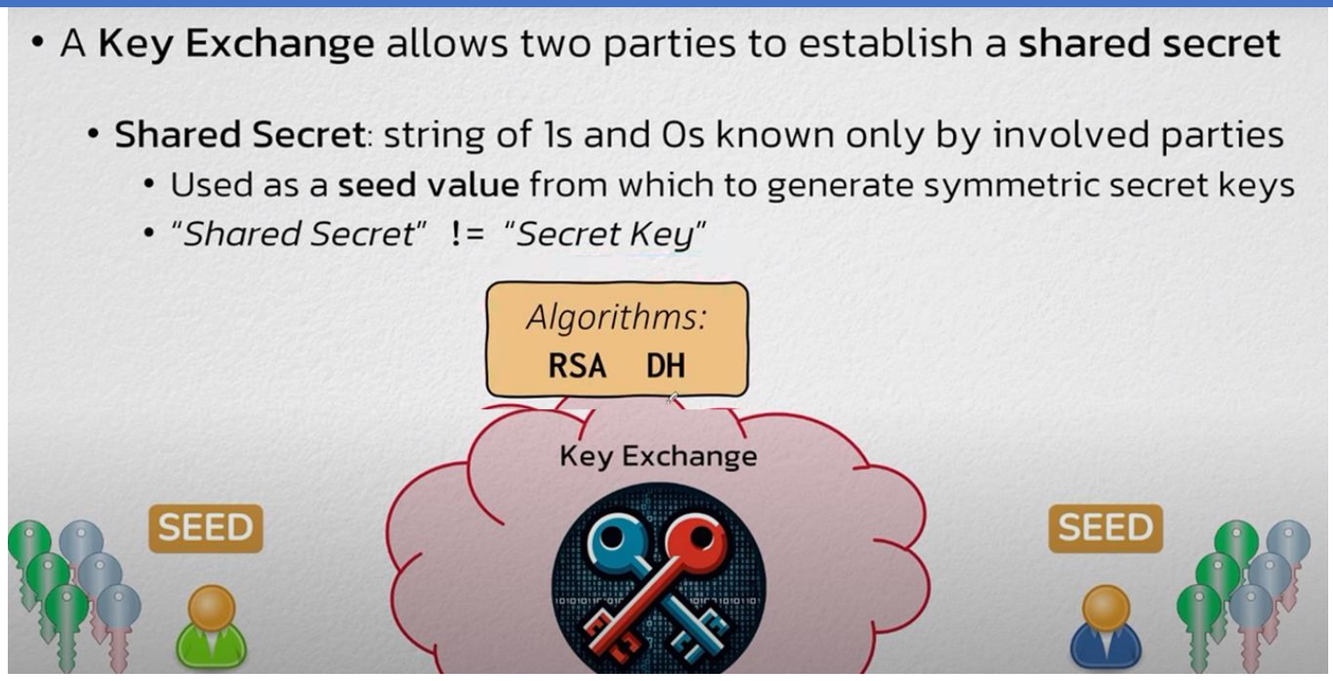
- DSA cannot do Encryption or Decryption or Key Exchanges
  - Digital *Signature* Algorithm
- Algorithm which involves two formulas:
  - **Signature Generation** & **Signature Verification**

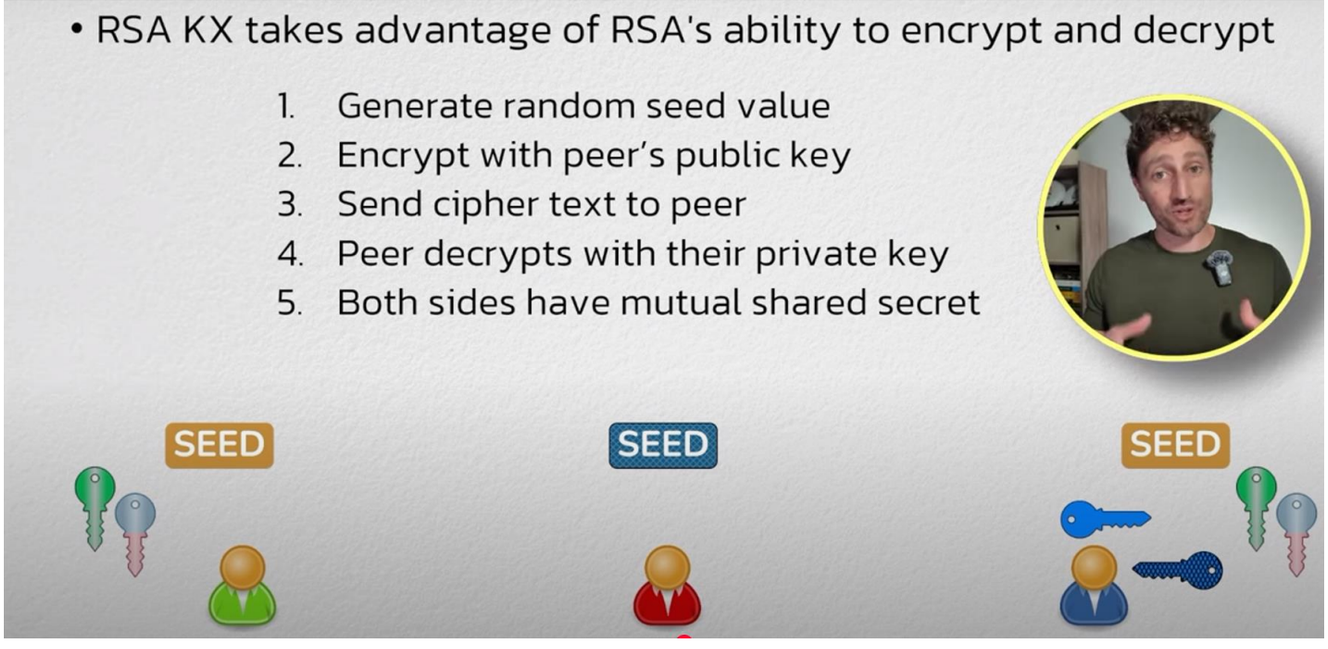
Signature Generation
Input: **Data**, **Private Key**
Output: **Signature**

Signature Verification
Input: **Data**, **Signature**, **Public Key**
Output: **1** or **0** (**True** or **False**)



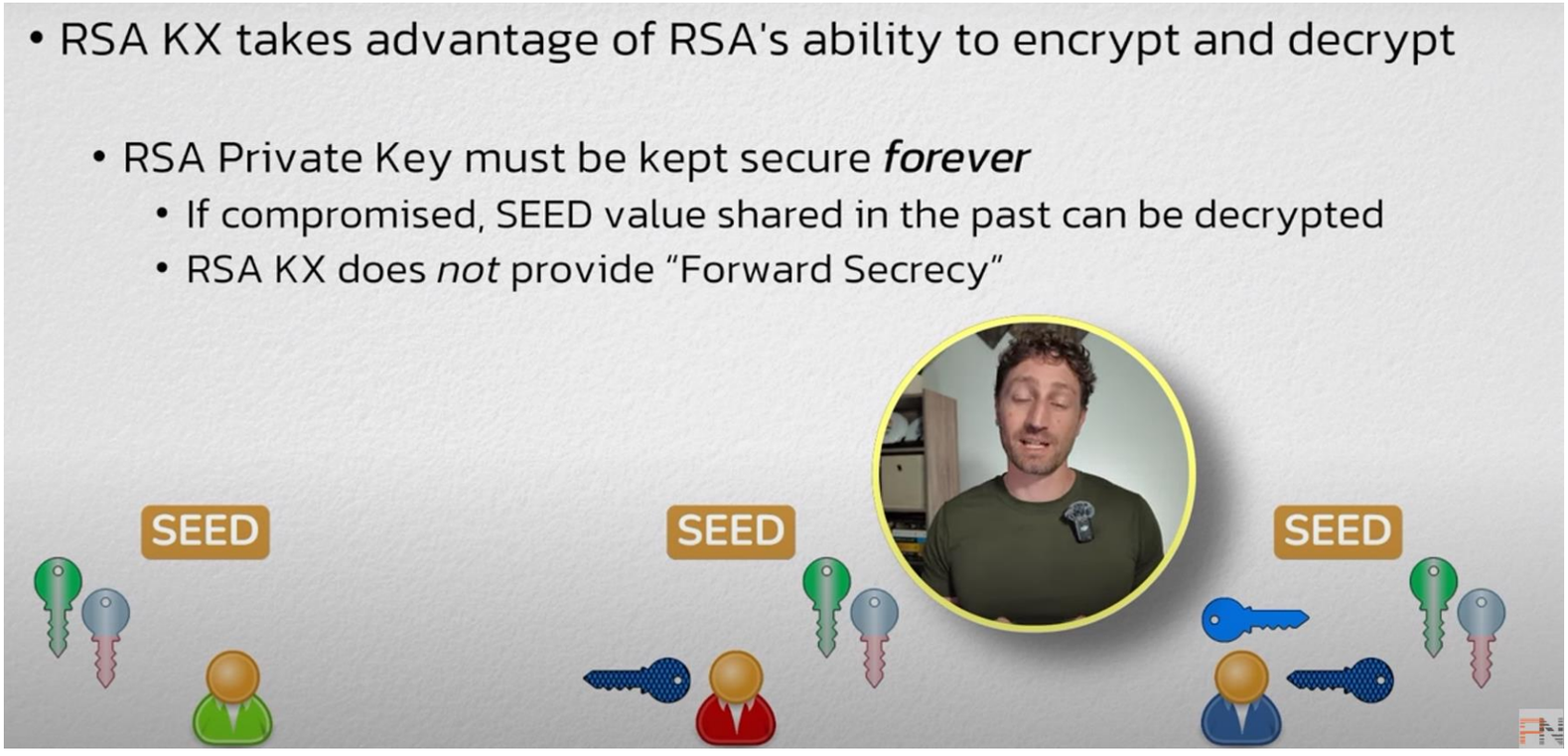
Key Exchange





Step: the server/user1 generates pair of key; the public is shared with others. User2 also generates a key called seed which is then encrypted using the public key of the user to whom he wants to send the data. User1 who has corresponding private key only decrypt the encrypted data. Now both parties, user1 and user2 can generate exact set of shared secret keys.

Key Exchange



If a private key is stolen, previously captured encrypted data can be decrypted, provided the same shared secret used for encryption is recovered

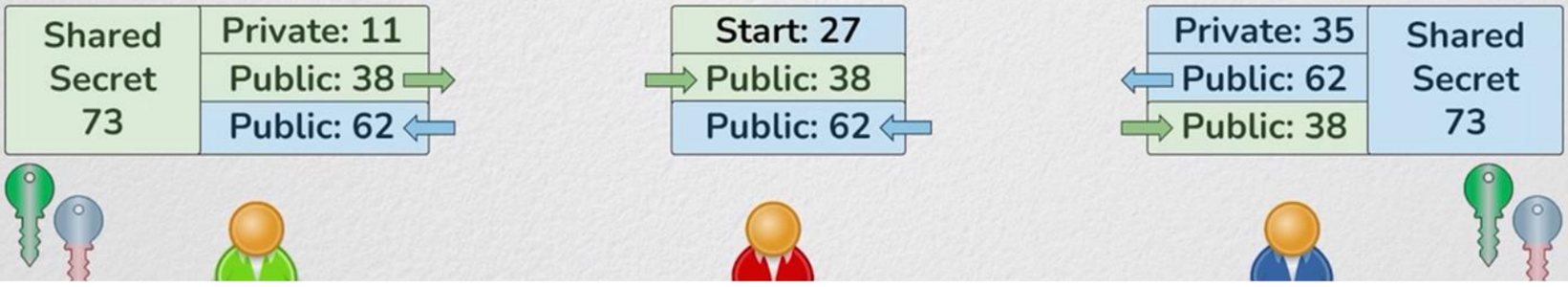
Diffie Hellman Key Exchange

- DH algorithm can only be used for a Key Exchange

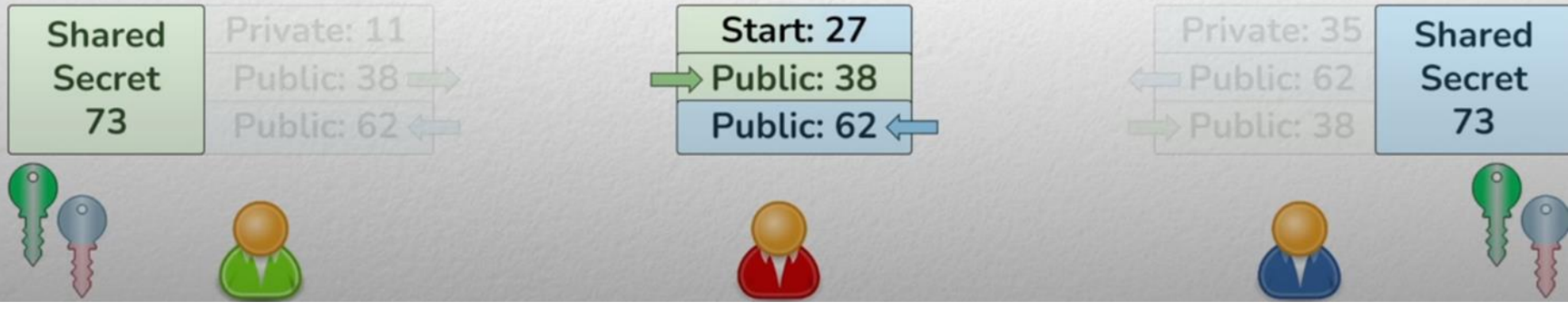
1. Peers publicly agree on a mutual starting value
2. Peers generate a Private value – never shared
3. Peers calculate a Public value – shared with peer
4. Combine Private value with peers' Public value

Note:
This is simplification

Real DH math uses Modular Exponentiation

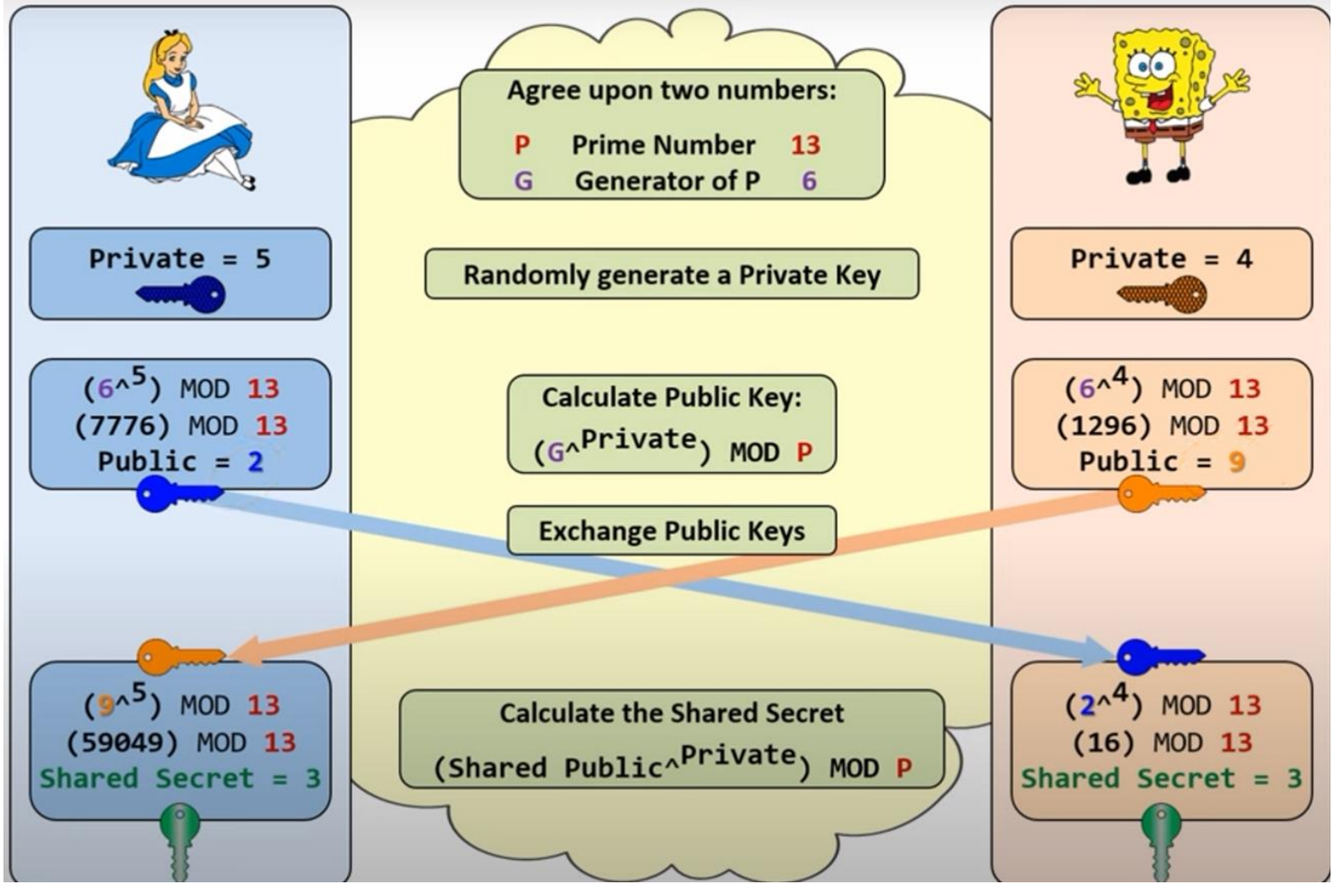


- Values used in Diffie-Hellman should be "*Ephemeral*"
  - Deleted after Shared Secret is attained
  - **Forward Secrecy** – Impossible to re-create Shared Secret in the future



Diffie Hellman Key Exchange

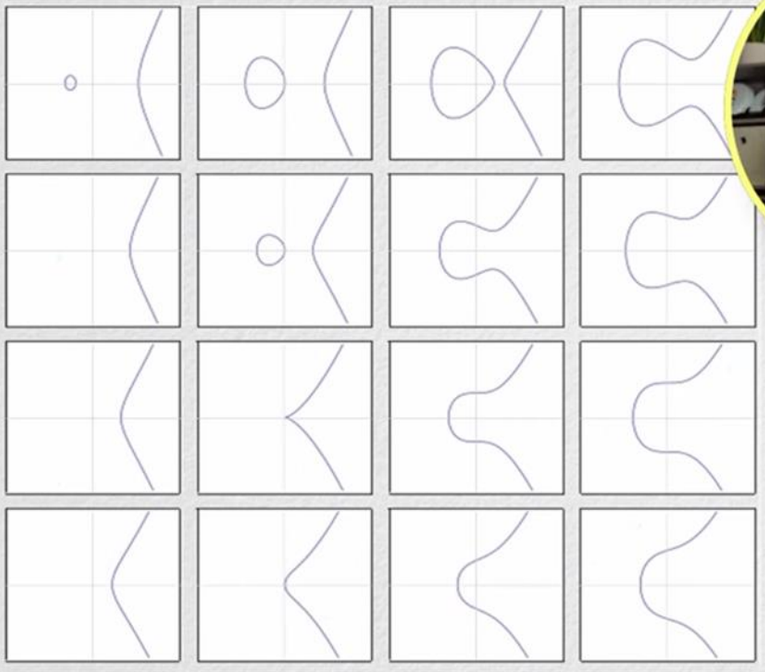
- Diffie-Hellman uses Modular Exponentiation
  - Exponentiation:
    - 7^2 = 7 * 7 = 49
    - 3^4 = 3 * 3 * 3 * 3 = 81
    - 9^5 = 9 * 9 * 9 * 9 * 9 = 59,049
  - Modular = Remainder Division
    - 49 mod 6 = 1
    - 81 mod 6 = 3
    - 59,049 mod 6 = 3



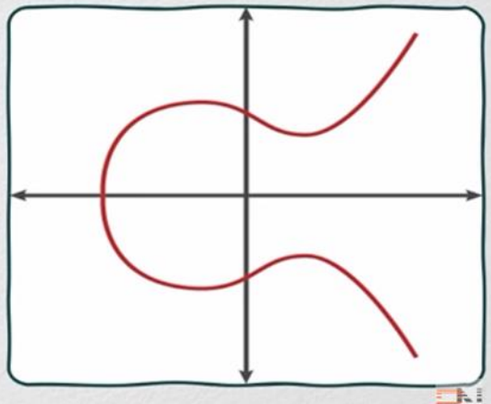
Elliptic Curve Cryptography

> Asymmetric Cryptography

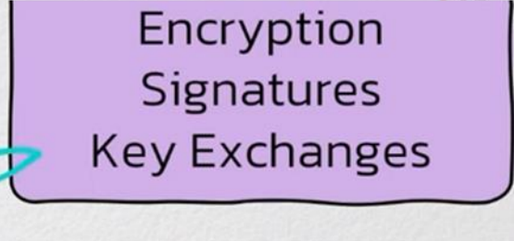
- Elliptic Curve Cryptography (ECC):

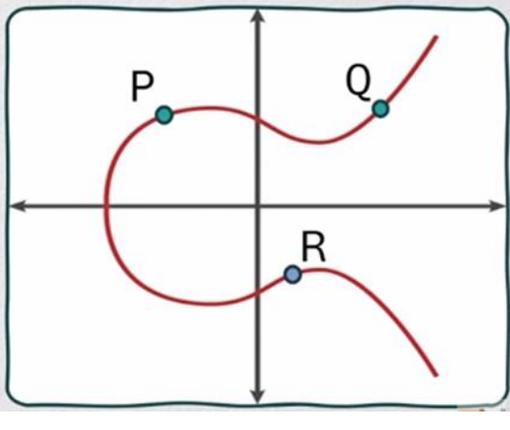


$$y^2 = x^3 + ax + b$$



- Elliptic Curve Cryptography (ECC):
  - Traditional crypto. uses *positive integers*
    - 2, 5, 37, 8954, 23462394, etc...
  - **ECC does the *same* math using points on an Elliptic Curve**
  - RSA, DSA, DH have Elliptic Curve variants:
    - **ECRSA** – rare, no security benefit
    - **ECDSA** – used & recommended
    - **ECDH** – used & recommended
  - ECC is **more secure** and **require smaller keys**





Elliptic Curve Cryptography

Security(in Bits)	RSA key length required	ECC key length required
80	1024	160-223
112	2048	224-255
128	3072	256-383
192	7680	384-511
256	15360	512+

1. ECDH – provides same level of security with smaller key sizes. Making more efficient in terms of bandwidth and computational resources.
2. Resistance – index calculate attacks can be used against DH.
3. Offering forward Secrecy – even if one side private key is compromised but the past data can’t be decrypted even if the packets are captured.